

JAVA DEVELOPER INTERNSHIP

TASK 3

COLLECTIONS AND EXCEPTION HANDLING

OBJECTIVE:

The objective of this task is to understand Java Collections Framework and Exception Handling concepts. The task demonstrates the use of ArrayList, HashSet, HashMap, try-catch blocks, and custom exceptions to manage data efficiently and handle runtime errors effectively.

TOPICS COVERED:

- ArrayList
- HashSet
- HashMap
- Try-Catch Block
- Custom Exception
- Exception Handling
- Java Collections Framework
- Runtime Error Handling
- Data Management

FEATURES IMPLEMENTED:

1. ArrayList Implementation

Used ArrayList to store and manage student names dynamically.

2. HashSet Implementation

Used HashSet to store course names and avoid duplicate entries.

3. HashMap Implementation

Used HashMap to associate student IDs with student names.

4. Exception Handling

Implemented try-catch blocks to handle runtime exceptions and prevent abnormal program termination.

5. Custom Exception

Created a user-defined exception to validate age and display appropriate error messages.

This task demonstrates the use of Java Collections Framework and Exception Handling mechanisms. The program efficiently manages data using ArrayList, HashSet, and HashMap, while try-catch blocks and custom exceptions ensure proper error handling and improve program reliability.

SOURCE CODE:

CollectionsAndExceptionDemo.java

```
import java.util.*;

class InvalidAgeException extends Exception

    public InvalidAgeException(String message)

{
    super(message);
}

}

public class CollectionsAndExceptionDemo

{
    static void checkAge(int age) throws InvalidAgeException

    {
        if (age < 18)

        {
            throw new InvalidAgeException("Age must be 18 or above.");
        }

        else

        {
            System.out.println("Age is valid.");
        }
    }
}
```

```
public static void main(String[] args)
{
    // ArrayList Example

    ArrayList<String> students = new ArrayList<>();

    students.add("Rahul");

    students.add("Priya");

    students.add("Kiran");

    System.out.println("ArrayList:");

    System.out.println(students);

    // HashSet Example

    HashSet<String> courses = new HashSet<>();

    courses.add("Java");

    courses.add("Python");

    courses.add("Java");

    System.out.println("\nHashSet:");

    System.out.println(courses);

    // HashMap Example

    HashMap<Integer, String> studentMap = new HashMap<>();

    studentMap.put(101, "Rahul");

    studentMap.put(102, "Priya");

    System.out.println("\nHashMap:");

    System.out.println(studentMap);
}
```

```

// Try-Catch Example

try
{
    int result = 10 / 0;

    System.out.println(result);
}

catch (ArithmeticException e)
{
    System.out.println("\nException Caught: " + e.getMessage());
}

// Custom Exception Example

try
{
    checkAge(15);
}

catch (InvalidAgeException e)
{
    System.out.println("Custom Exception: " + e.getMessage());
}

}

```

The CollectionsAndExceptionDemo program demonstrates the use of Java Collections Framework and Exception Handling mechanisms. ArrayList is used to store student names dynamically, HashSet is used to store unique course names, and HashMap is used to associate student IDs with student names. The program also implements try-catch blocks to handle runtime exceptions and defines a custom exception to validate age and display appropriate error messages.

OUTPUT:



```
C:\Windows\System32\cmd.exe
Microsoft Windows [Version 10.0.26200.8524]
(c) Microsoft Corporation. All rights reserved.

C:\Users\DEEKSHITH\OneDrive\Pictures\Documents\Desktop\srihani\Task-3-Collections-and-Exception-Handling>javac CollectionsAndExceptionDemo.java

C:\Users\DEEKSHITH\OneDrive\Pictures\Documents\Desktop\srihani\Task-3-Collections-and-Exception-Handling>java CollectionsAndExceptionDemo

ArrayList:
[Rahul, Priya, Kiran]

HashSet:
[Java, Python]

HashMap:
{101=Rahul, 102=Priya}

Exception Caught: / by zero
Custom Exception: Age must be 18 or above.
```

The program was executed successfully and demonstrated the use of ArrayList, HashSet, and HashMap for efficient data management. Exception handling using try-catch blocks and custom exceptions was also implemented successfully to handle runtime errors and improve program reliability.

CONCEPTS USED:

ArrayList

ArrayList is a dynamic collection class used to store and manage data efficiently. It allows elements to be added and removed dynamically during program execution.

HashSet

HashSet is used to store unique elements and automatically eliminates duplicate values. It provides efficient data retrieval and storage.

HashMap

HashMap stores data in key-value pairs, allowing fast access to values based on their corresponding keys. It is useful for mapping relationships between data elements.

Exception Handling

Exception handling is a mechanism used to handle runtime errors and prevent abnormal program termination. It improves the reliability and stability of applications.

Try-Catch Block

The try-catch block is used to catch and handle exceptions that occur during program execution. It ensures smooth execution of the program even when errors occur.

Custom Exception

Custom exceptions are user-defined exceptions created to handle specific error conditions. They provide meaningful error messages and improve program maintainability.

Java Collections Framework

The Java Collections Framework provides classes and interfaces for storing and manipulating groups of data efficiently. It simplifies data management and enhances program performance.

CONCLUSION:

This task provided practical experience in using the Java Collections Framework and Exception Handling mechanisms. Through the implementation of ArrayList, HashSet, HashMap, try-catch blocks, and custom exceptions, efficient data management and error handling techniques were successfully demonstrated. The task enhanced problem-solving skills and strengthened the understanding of Java programming concepts used in developing reliable and maintainable applications.